HDL VERIFICATION AND METHODOLOGIES

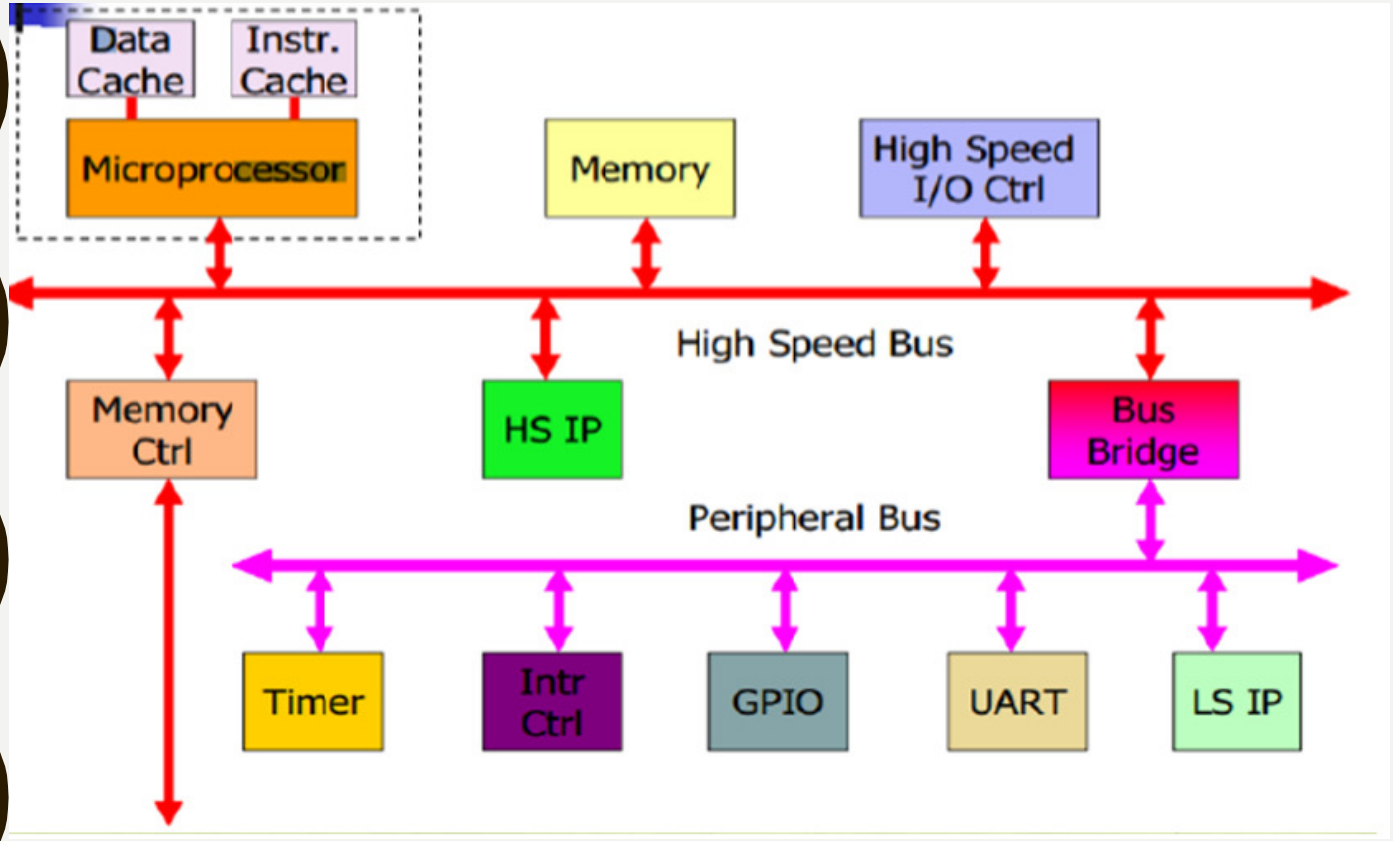
COURSE OBJECTIVES

- This course teaches
  - Design verification concepts
  - Coding in System Verilog language for verification
  - Verification Methodology (UVM,OVM)

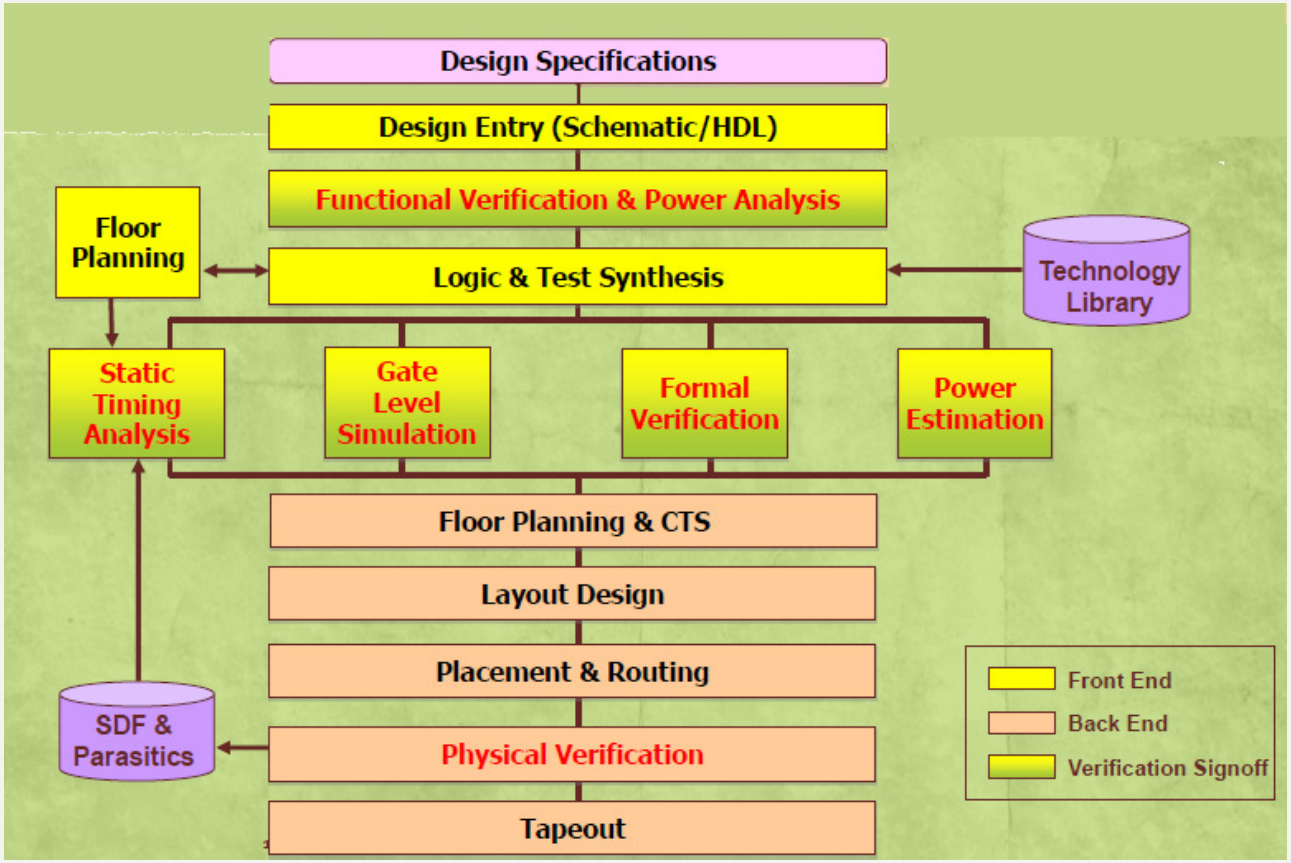
SYLLABUS OUTLINE

- Unit I – about Verification importance and various verification approaches
- Unit II – about Verification plan and various code & functional coverage strategies
- Unit III – about systemVerilog programming language
- Unit IV – about various HDL verification methodologies using systemVerilog
- Unit V – about Universal Verification Methodology (UVM) using systemVerilog

SYSTEM-ON-CHIP



VLSI/CHIP DESIGN FLOW



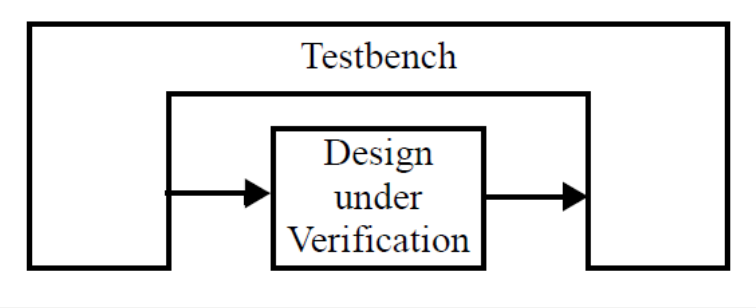
UNIT-I :
IMPORTANCE OF VERIFICATION AND DIFFERENT APPROACHES

CONCEPTS OF VERIFICATION

- WHAT IS VERIFICATION?
  - Verification is not a testbench, nor is it a series of testbenches.
  - Verification is a process used to demonstrate that the intent of a design is preserved in its implementation.
  - We all perform verification processes throughout our daily lives:
    - balancing a checkbook
    - tasting a cooking dish
    - associating landmarks with symbols on a map.
  - verification also plays a key role to design reuse

CONCEPTS OF VERIFICATION

- WHAT IS A TESTBENCH?
  - The term "testbench" usually refers to simulation code used to create a predetermined input sequence to a design, then optionally to observe the response.
  - Testbenches are implemented using System-Verilog, but they may also include external data files or C routines.



- Figure shows how a testbench interacts with a design under verification (DUV).
- The testbench provides inputs to the design and watches any outputs.
- Notice how this is a completely closed system - no inputs or outputs go in or out.
- The testbench is effectively a model of the universe as far as the design is concerned.
- The verification challenge is to determine what input patterns to supply to the design and what is the expected output of a properly working design when submitted to those input patterns.

IMPORTANCE OF VERIFICATION

- 70% of design effort goes to verification.
  - Today, in the era of multi-million gate ASICs and FPGAs, reusable intellectual property (IP), and system-on-a-chip (SoC) designs, verification consumes about 70% of the design effort.
  - Design teams, properly staffed to address the verification challenge, include engineers dedicated to verification.
  - The number of verification engineers can be up to twice the number of RTL designers.
- Verification is on the critical path.
  - Given the amount of effort demanded by verification, the shortage of qualified hardware design and verification engineers, and the quantity of code that must be produced, it is no surprise that, in all projects, verification rests squarely in the critical path.
  - The fact that verification is often considered after the design has been completed, when the schedule has already been ruined, compounds the problem.
  - It is also the reason verification is the target of the most recent tools and methodologies.
  - These tools and methodologies attempt to reduce the overall verification time by enabling parallelism of effort, higher abstraction levels and automation.

IMPORTANCE OF VERIFICATION

- Verification time can be reduced through parallelism.
  - If efforts can be parallelized, additional resources can be applied effectively to reduce the total verification time.
  - For example, digging a hole in the ground can be parallelized by providing more workers armed with shovels.
  - To parallelize the verification effort, it is necessary to be able to write and debug testbenches in parallel with each other as well as in parallel with the implementation of the design.
- Verification time can be reduced through abstraction.
  - Providing higher abstraction levels enables you to work more efficiently without worrying about low-level details.
  - Using a backhoe to dig the same hole mentioned above is an example of using a higher abstraction level.

IMPORTANCE OF VERIFICATION

- Using abstraction reduces control over low-level details.
  - Higher abstraction levels are usually accompanied by a reduction in control and therefore must be chosen wisely.
  - These higher abstraction levels often require additional training to understand the abstraction mechanism and how the desired effect can be produced.
  - If a verification process uses higher abstraction levels by working at the transaction- or bus-cycle levels (or even higher ones), it will be easier to create large amount of stimulus.
  - But it may be difficult to produce a specific sequence of low-level zeroes and ones.
  - Similarly, using a backhoe to dig a hole suffers from the same loss-of control problem: The worker is no longer directly interacting with the dirt;
  - instead the worker is manipulating levers and pedals.
  - Digging happens much faster, but with lower precision and only by a trained operator.

IMPORTANCE OF VERIFICATION

- Lower levels of abstraction must remain visible.
  - It may be necessary to navigate between levels of abstraction.
  - Verification can be accomplished using a bottom-up approach where the interface blocks and physical level details are verified first.
  - The protocol levels can then be verified without having to worry about the physical signals.
  - Verification can also be accomplished using a top-down approach where the protocol-level functionality is verified first using a transaction-level model without any physical-level interfaces.
  - The details of the physical transport mechanisms can then be added later.
  - The transition between levels of abstraction may also occur dynamically during the execution of a testbench.
  - The testbench may generally work at a high-level of abstraction, verifying the correctness of protocol-level operations.
  - The testbench can then switch to a lower level of abstraction to inject a physical-level parity error to verify that protocol-level operations remain unaffected.
  - Similarly, the backhoe operator can let a worker jump into the hole and use a shovel to uncover a gas pipeline

IMPORTANCE OF VERIFICATION

- Verification time can be reduced through automation.
  - Automation lets you do something else while a machine completes a task autonomously, faster and with predictable results.
  - Automation requires standard processes with well-defined inputs and outputs.
  - Not all processes can be automated.
  - For example, holes must be dug in a variety of shapes, sizes, depths, locations and in varying soil conditions, which render general-purpose automation impossible.
  - Verification faces similar challenges.
  - Because of the variety of functions, interfaces, protocols and transformations that must be verified, it is not possible to provide a general purpose automation solution for verification, given today's technology.
  - It is possible to automate some portion of the verification process, especially when applied to a narrow application domain.
  - Tools automating various portions of the verification process are being introduced.
  - For example, there are tools that will automatically generate bus-functional models from a higher-level abstract specification.
  - Similarly, trenchers have automated digging holes used to lay down conduits or cables at shallow depths.

IMPORTANCE OF VERIFICATION

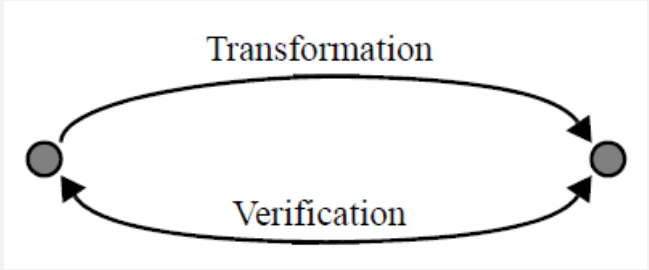
- Randomization can be used as an automation tool.
  - For specific domains, automation can be emulated using randomization.
  - By constraining a random generator to produce valid inputs within the bounds of a particular domain, it is possible to automatically produce almost all of the interesting conditions.
  - This type of automation process takes more computation time to achieve the same result, but it is completely autonomous, freeing valuable resources to work on other critical tasks.
  - Furthermore, this process can be parallelized easily by concurrently running several random generators.
  - They can also operate overnight, increasing the total number of productive hours.

RECONVERGENCE MODEL

- The reconvergence model is a **conceptual representation of the verification process**.
- It is used to **illustrate what exactly is being verified**.
- Do you know **what you are actually verifying**?
  - One of the most important questions you must be able to answer is: "**What are you verifying?**"
  - The purpose of verification is to ensure that the **result of some transformation is as intended** or as expected.
  - For example, the purpose of balancing a checkbook is to ensure that all transactions have been recorded accurately and confirm that the balance in the register reflects the amount of available funds.

RECONVERGENCE MODEL

- Verification is the **reconciliation, through different means**, of a specification and an output.
  - Figure shows that verification of a transformation can be accomplished only through a second reconvergent path with a common source.
  - The transformation **can be any process that takes an input and produces an output**.
  - RTL coding from a specification, insertion of a scan chain, synthesizing RTL code into a gate-level netlist and layout of a gate-level netlist are some of the transformations performed in a hardware design project.
  - The verification process **reconciles the result with the starting point**.
  - If there is **no starting point** common to the transformation and the verification, ***no* verification takes place**.



Reconvergent paths in verification

RECONVERGENCE MODEL

- Balancing a checkbook is a verification process.
  - The reconvergence model can be described using the checkbook example as illustrated in Figure.
  - The common origin is the previous month's balance in the checking account.
  - The transformation is the writing, recording and debiting of several checks during a one-month period.
  - The verification reconciles the final balance in the checkbook register using this month's bank statement.

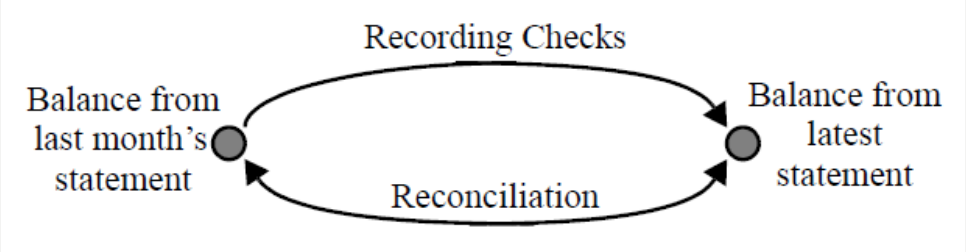
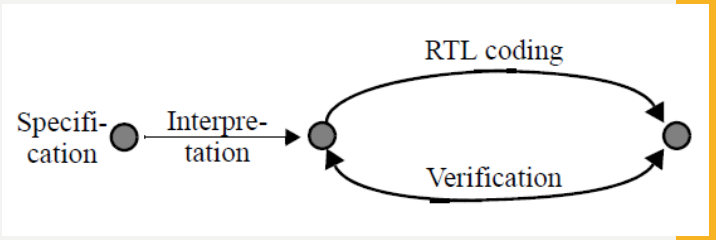


Illustration of Balancing a checkbook

RECONVERGENCE MODEL - THE HUMAN FACTOR

- Verifying your own design **verifies against your interpretation, not against the specification**.
  - If the transformation process is not completely automated from end to end, it is necessary for an individual (or group of individuals) to interpret a specification of the desired outcome and then perform the transformation.
  - RTL coding is an example of this situation.
  - A design team interprets a written specification document and **produces what they believe to be functionally correct** synthesizable HDL code.
  - Usually, each engineer is left to verify that the code written is indeed functionally correct.

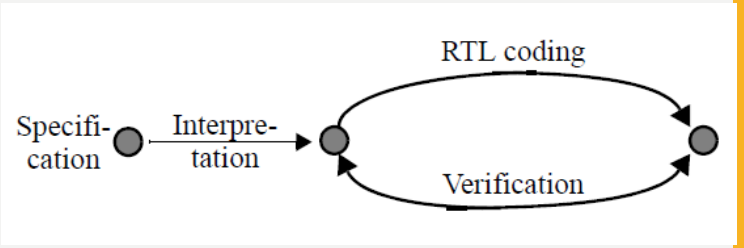


Reconvergent paths in ambiguous situation

RECONVERGENCE MODEL - THE HUMAN FACTOR

Verifying your own design verifies against your interpretation, not against the specification.

- Figure shows the reconvergence model of the situation described above.
- RTL coding requires interpretation of a written specification.
- If the **same person performs the verification, then the common origin is that interpretation, not the specification**.
- In this situation, the verification effort verifies whether the design accurately represents the implementer's interpretation of that specification.
- **If that interpretation is wrong** in any way, then this **verification activity will never highlight it**.
- Any human intervention in a process is a source of uncertainty and unrepeatability.
- The probability of **human-introduced errors in a process can be reduced through several complementary mechanisms: automation, mistake-proofing, redundancy.**



Reconvergent paths in ambiguous situation

RECONVERGENCE MODEL - THE HUMAN FACTOR

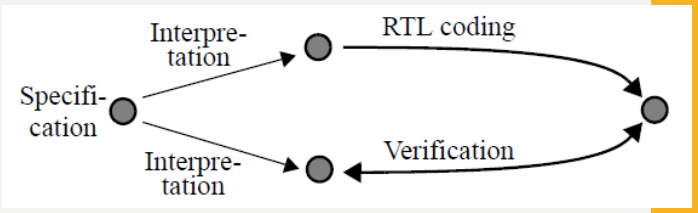
- Automation - Eliminate human intervention
  - Automation is the obvious way to **eliminate human-introduced errors in a process**.
  - Automation **takes human intervention completely out of the process.**
  - However, automation is **not always possible**, especially **in processes that are not well-defined** and continue to require human ingenuity and creativity, such as hardware design.

RECONVERGENCE MODEL - THE HUMAN FACTOR

- Poka-Yoke (Japanese term meaning mistake-proofing) - Make human intervention foolproof
  - Another possibility is to mistake-proof the human intervention by ***reducing it to simple, and foolproof steps***.
  - Human intervention is needed ***only to decide on the particular sequence or steps*** required to obtain the desired results.
  - This mechanism is also known as ***poka-yoke in Total Quality Management circles***.
  - It is usually the ***last step toward complete automation of a process***.
  - ***However, just like automation, it requires a well-defined process with standard transformation steps***.
  - The verification process remains an art that, to this day, does not lend itself to fool-proof steps.

RECONVERGENCE MODEL - THE HUMAN FACTOR

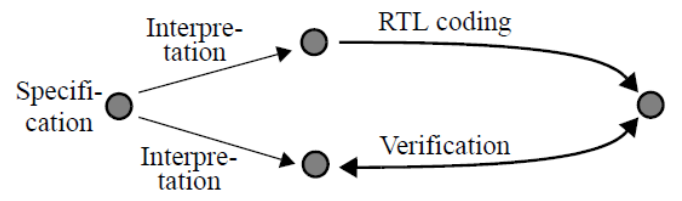
- Redundancy - Have two individuals check each other's work
  - The ***final alternative*** to removing human errors is redundancy.
  - It is the ***simplest, but also the most costly*** mechanism.
  - Redundancy requires every transformation path to be ***duplicated***.
  - Every transformation accomplished by a human is either ***independently verified by another individual***, or two complete and **separate transformations are performed** with each outcome compared to verify that both produced the same or equivalent output.
  - This mechanism is ***used in high-reliability environments***, such as airborne and space systems.
  - It is also used in industries where later redesign and replacement of ***a defective product would be more costly than the redundancy*** itself, such as ASIC design.



Redundancy in an ambiguous situation enables verification

RECONVERGENCE MODEL - THE HUMAN FACTOR

- A different person should verify
  - Figure shows the reconvergence model where ***redundancy is used to guard against misinterpretation of an ambiguous specification document***.
  - When used in the context of hardware design, where the transformation process is writing RTL code from a written specification document, this mechanism implies that a ***different individual must verify that implementation***.



Redundancy in an ambiguous situation enables verification

FORMAL VERIFICATION

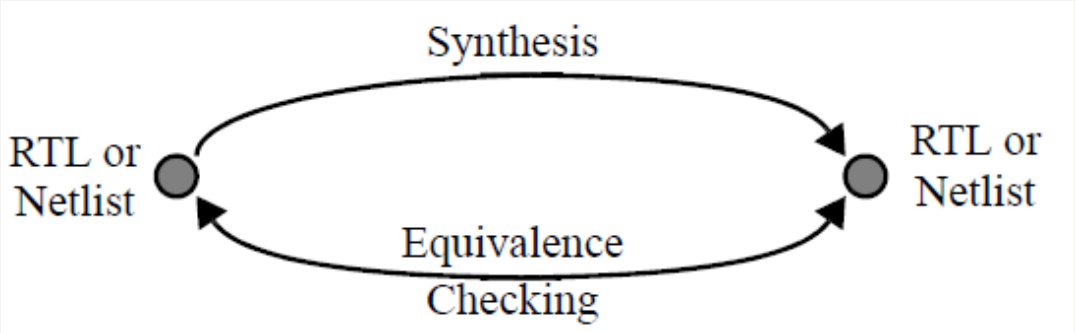
- WHAT IS BEING VERIFIED?
  - Choosing the **common origin and reconvergence points** determines what is being verified.
  - These origin and reconvergence points are often **determined by the tool used to perform the verification**.
  - It is important to understand where these points lie **to know which transformation is being verified.**
  - Formal verification, property checking, functional verification, and rule checkers **verify different things because they have different origin and reconvergence points**.

FORMAL VERIFICATION

- Formal verification **does not eliminate the need to write testbenches**.
  - Formal verification is often misunderstood initially.
  - Engineers unfamiliar with the formal verification process often imagine that it is a tool that mathematically determines whether their design is correct, without having to write testbenches.
  - Once they understand what the end points of the formal verification reconvergent paths are, you know what exactly is being verified.
  - The application of formal verification falls under **two broad categories: equivalence checking and property checking**.

EQUIVALENCE CHECKING

- Equivalence checking **compares two models**
  - Figure shows the reconvergence model for equivalence checking.
  - This formal verification process **mathematically proves that the origin and output are logically equivalent** and that **the transformation preserved its functionality.**
- It can compare two netlists
  - In its most common use, equivalence checking compares two netlists to **ensure that some netlist post-processing**, such as scan chain insertion, clock-tree synthesis or manual modification, **did not change the functionality of the circuit**.



EQUIVALENCE CHECKING

- It **can detect bugs in the synthesis**
  - Another popular use of equivalence checking is **to verify that the netlist correctly implements the original RTL code**.
  - If one trusted the synthesis tool completely, this verification would not be necessary.
  - However, synthesis tools are large software systems that **depend on the correctness of algorithms and library information**.
  - History has shown that such systems are prone to error.
  - Equivalence checking is **used to keep the synthesis tool honest**.
  - In some rare instances, this form of equivalence checking is used to **verify that manually written RTL code faithfully represents a legacy gate-level design**.
  - Equivalence checking can also verify that **two RTL descriptions are logically identical**.
  - Proving their equivalence **avoids running lengthy regression simulations** when only minor non-functional changes are made to the source code to obtain better synthesis results.
  - Modern equivalence checkers can even deal with **sequential differences between RTL models**, such as re-architected FSMs or data pipelines

EQUIVALENCE CHECKING

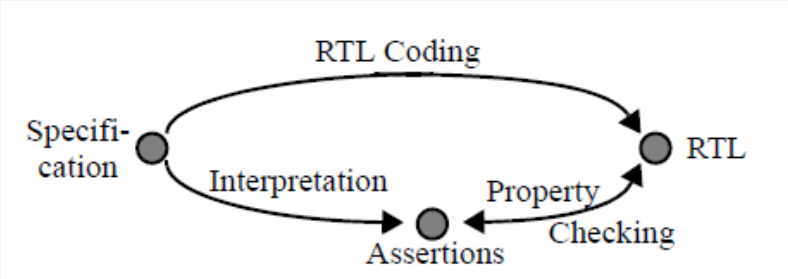
- Equivalence checking found a bug in an arithmetic operator.
  - Equivalence checking is a **true alternative path to the logic synthesis transformation** being verified.
  - It is only interested in comparing Boolean and sequential logic functions, **not mapping these functions to a specific technology** while meeting stringent design constraints.
  - Engineers using **equivalence checking found a design at Digital Equipment Corporation (now part of HP**) to be synthesized incorrectly.
  - The design used a synthetic operator that was **functionally incorrect when handling more than 48 bits**.
  - To the synthesis tool's defense, **the documentation of the operator clearly stated that correctness was not guaranteed above 48 bits**.
  - Since the **synthesis tool had no knowledge of documentation**, it could not know it was generating invalid logic.
  - Equivalence checking **quickly identified a problem that could have been very difficult to detect using gate-level simulation**.

PROPERTY CHECKING

- Property checking **proves assertions about the behavior of the design**.
  - Property checking is a more recent application of formal verification technology.
  - In it, assertions or characteristics of a design are **formally proven or disproved**.
  - For example, all state machines in a design could be checked for unreachable or isolated states.
  - A more powerful property checker may be able to **determine if deadlock conditions can occur**.
  - Another type of assertion that can be formally verified relates to **interfaces**.
  - Using the **SystemVerilog property specification language**, **assertions about the interfaces** of a design are stated and the tool attempts to prove or disprove them.

PROPERTY CHECKING

- Assertions must not be trivial (obvious).
  - The reconvergence model for property checking is shown in Figure.
  - The **greatest obstacle** for property checking technology is identifying, through interpretation of the design specification, **which assertions to prove**.
  - Of those assertions, **only a subset can be proven feasibly**.
  - Furthermore, **for a proof to be useful, assertions must not be trivial restatements** of the behavior already captured by the RTL code.
  - They **should be based on external requirements** that the design must meet.



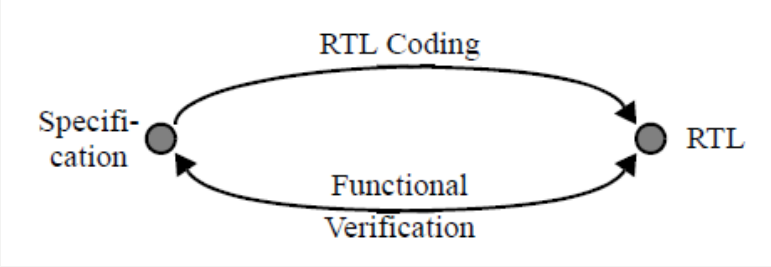
Property checking paths

PROPERTY CHECKING

- Assertions are **good at checking temporal relationships of synchronous signals**.
  - Current technology **has difficulties proving high-level assertions** about a design to ensure that complex functionality is correctly implemented.
  - It would be nice to be able to concisely assert that, given specific register settings, a sequence of packets will end up at a set of outputs in some relative order.
  - Unfortunately, there are **two obstacles** in front of this goal.
    - First, **property checking technology is limited in its capacity** to deal with complex designs.
    - Second, the assertion languages with formal semantics **can efficiently describe cycle-based temporal relationships only** between low-level signals and simple transformations – a very big hierarchical complex design difficult to assert at low level.
  - Their inherent RTL or cycle-based nature makes **it difficult to state high-level transformation properties**.

FUNCTIONAL VERIFICATION

- Functional verification verifies design intent
  - The main purpose of functional verification is **to ensure that a design implements intended functionality.**
  - As shown by the reconvergence model in Figure, functional verification reconciles a design with its specification.
  - Without functional verification,
    - one must trust that the transformation of a specification document into RTL code was performed correctly,
    - without misinterpretation of the specification's intent.



Functional verification paths

FUNCTIONAL VS FORMAL VERIFICATION

- Functional verification is the task of **verifying that the logic design conforms to specificatio**n
  - attempts to answer the question "**Does this proposed design do what is intended?**"

- Formal verification attempts to **prove mathematically that certain requirements are met**, or that certain undesired behaviors (such as deadlock) cannot occur.
  - the act of **proving or disproving the correctness of intended algorithms (**underlying a system) **using formal methods of mathematics**.

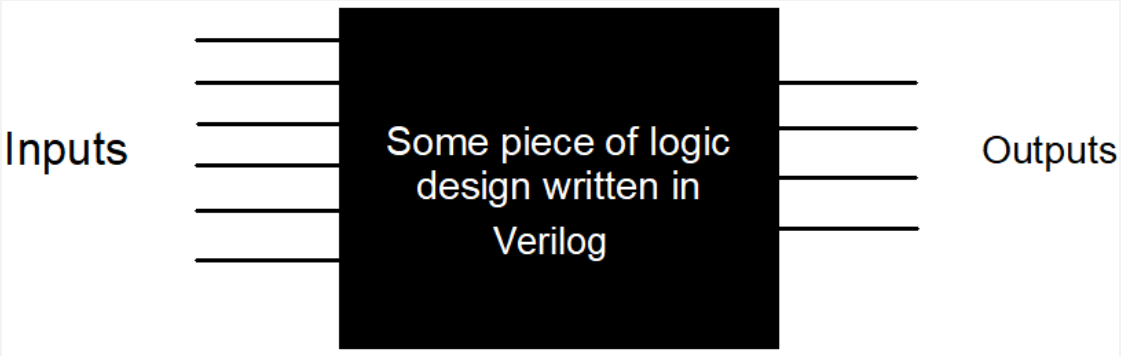
FUNCTIONAL VERIFICATION

- You can prove the presence of bugs, but you cannot prove their absence.
  - unless a specification is written in a formal language with **precise semantics** - it is impossible to prove that a design meets the intent of its specification.
  - Specification documents are written using natural languages by individuals with **varying degrees of ability in communicating their intentions**.
  - Any document is open to interpretation.
  - One **can easily prove that the design does not implement the intended function** by identifying a single discrepancy.
  - The converse, sadly, is not true:
  - **No one can prove that there are no discrepancies.**
  - Functional verification, as a process, **can show that a design meets the intent of its specification - but it cannot prove it**.

FUNCTIONAL VERIFICATION APPROACHES

- Black-Box Approach
- White-Box Approach
- Grey-Box Approach

BLACK-BOX



- The black box **has inputs, outputs, and performs some function**.
- The function **may be well documented...or not**.
- To verify a black box, you the **need to understand the function and be able to predict** outputs based on the inputs.
- The black box **can be a full system, a chip, a unit of a chip, or a single macro.**

BLACK-BOX

- Black-box verification cannot look at or know about the inside of a design.
  - With a black-box approach, functional verification is **performed without any knowledge of the actual implementation of a design**.
  - All verification is accomplished through the available interfaces, **without direct access to the internal state of the design**, without knowledge of its **structure and implementation**.
  - This method suffers from an obvious **lack of visibility and controllability**.
  - It is often **difficult** to set up an interesting state combination or **to isolate some functionality**.
  - It is equally **difficult to observe the response** from the input and **difficult locate the source of the problem**.
  - This difficulty arises from the frequent long delays between the occurrence of a problem and the appearance of its symptom on the design's outputs.

BLACK-BOX

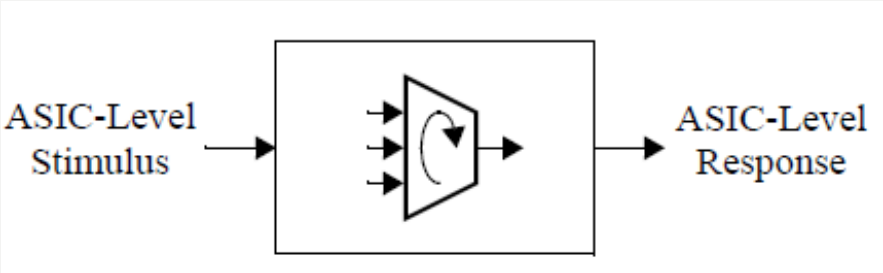
- Testcase is independent of implementation
  - The **advantage** of black-box verification is that it **does not depend on any specific implementation**.
  - Whether the **design is implemented** in a single ASIC, RTL code, transaction-level model, gates, multiple FPGAs, a circuit board or entirely in software, **is irrelevant**.
  - A black-box functional verification approach forms a true conformance verification that can be used to show that a particular design **implements the intent of a specification, regardless of its implementation.**
  - A set of **black-box testbenches** can be developed on a transaction-level model of the design and run, unmodified, on the RTL model of the design to demonstrate that they **are equivalent**.
  - Black-box testbenches **can be used as a set of golden testbenches**.

BLACK-BOX

- In black-box verification, it is difficult to control and observe specific features.
  - The pure black-box approach is impractical in today's large designs.
  - A multi-million gates ASIC possesses too many internal signals and states to effectively verify all of its functionality from its periphery.
  - Critical functions, deep into the design, will be difficult to control and observe.

BLACK-BOX

- Furthermore, a design fault may not readily present symptoms of a flaw at the outputs of the ASIC.
- For example, the black-box ASIC-level testbench in Figure is used to verify a critical round-robin arbiter.
- If the arbiter is not completely fair in its implementation, what symptoms would be visible at the outputs?
- This type of fault could only be found through performance analysis
  - using several long simulations to identify discrepancies
  - between the actual throughput of a channel compared with its theoretical throughput.
- And a two percent discrepancy in a channel throughput in three overnight simulations can be explained so easily as a simple statistical error...



Black-box verification of a low-level feature

WHITE-BOX

- White box verification means that **the internal facilities are visible** and utilized by the testbench stimulus.
- Examples: Unit/Module level verification

WHITE BOX

- White box verification **has intimate knowledge and control of the internals of a design**.
  - As the name suggests, a white-box approach **has full visibility and controllability of the internal structure and implementation** of the design being verified.
  - This method has the **advantage of being able to set up an interesting combination of states and inputs quickly**, or to isolate a particular function.
  - It can then **easily observe the results as the verification progresses** and **immediately report any discrepancies** from the expected behavior.

WHITE BOX

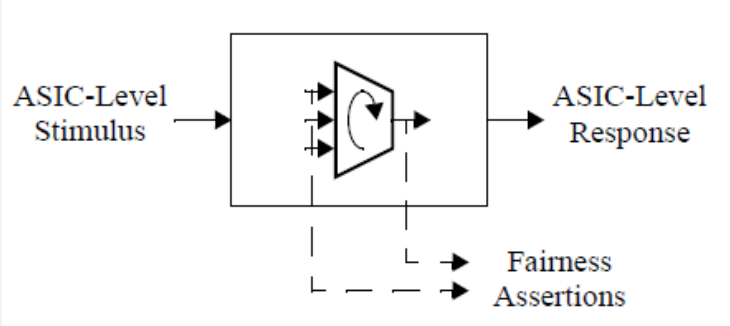
- White-box verification is **tied to a specific implementation**.
  - However, this approach is **tightly integrated with a particular implementation**.
  - Changes in the design **may require changes in the testbench**.
  - Furthermore, those testbenches **cannot be used in gate-level simulations**, on alternative implementations or future redesigns.
  - It **also requires detailed knowledge of the design implementation** to know which significant conditions to create and which results to observe.

WHITE BOX

- White-box techniques can augment black-box approaches.
  - White-box verification is **a useful complement to black-box verification**.
  - This approach **can ensure that low-level implementation**.
  - Specific features behave properly, **such as counters rolling over** after reaching their end count value or **datapaths being appropriately steered and sequenced**.
  - The white-box approach **can be used only to verify the correctness of the functionality**, while **still relying on the black- or grey-box stimulus.**
  - **Assertions are ideal for implementing** white-box checks in RTL code.

WHITE BOX

- White-box techniques **can augment black-box approaches**.
  - Assertions are ideal for implementing white-box checks in RTL code.
  - For example, below figure shows the black-box ASIC-level environment augmented with assertions to verify the functional correctness of the round-robin arbiter.
  - Should fairness **not be implemented correctly, the white-box checks would immediately report a failure**.
  - The reported error **would also make it easier to identify and confirm the cause of the problem**, compared to a two percent throughput discrepancy.



White-box checks in black-box environment

WHITE BOX

- Checkered-box is used in system-level verification..
  - A checkered-box verification approach is often used on SoC design and system-level verification.
  - A system is defined as a design composed of independently designed and verified components.
  - The objective of system-level verification is to verify the system-level features, not reverify the individual components.
  - Because of the large number of possible states and the difficulty in setting up interesting conditions, system-level verification is often accomplished by **treating it as a collection of black-boxes.**
  - The independently designed components are treated as black-boxes, **but the system itself is treated as a white-box, with full controllability and observability**.

GREY-BOX

- Grey box verification means that a **limited number of facilities are utilized in a mostly black-box environment**.
- Example: **Most environments!**
- Prediction of correct results on the interface is **occasionally impossible without viewing an internal signal**.

GREY-BOX VERIFICATION

- Grey-box verification is a compromise between the independency of black-box verification and the dependence on the implementation of white-box verification.
- Testcases may not be relevant on another implementation.
  - grey-box approach **controls and observes a design entirely through its top-level interfaces** – just like in black-box verification,.
  - However, the particular verification being accomplished is intended **to exercise significant features specific to the implementation**.
  - The same verification of a different implementation would be successful, but the verification may not be particularly more interesting than any other black-box verification.
  - A typical grey-box test case is one written to **increase coverage metrics**.
  - The input stimulus is designed **to execute specific lines of code** or create a specific set of conditions in the design.
  - Should the **structure of the design change** (but not the function), this test case, while still correct, **may no longer contribute toward better coverage**.

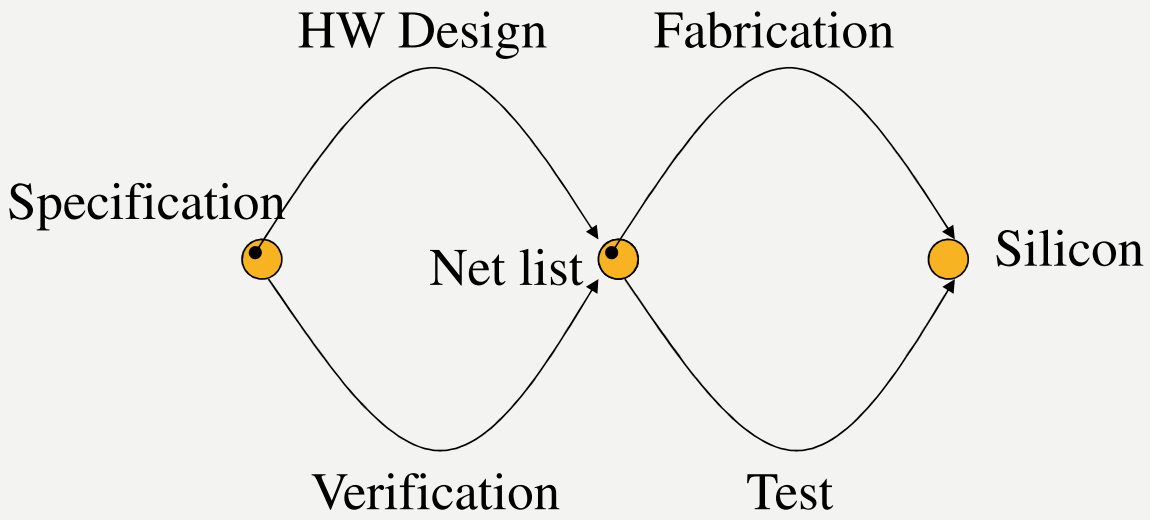
PERFECT VERIFICATION

- Prediction of correct results on the interface is **occasionally impossible without viewing an internal signal**.
- To fully verify a black-box, you **must show that the logic works correctly for all combinations of inputs**.
  - This entails:
    - **Driving all permutations on the input lines**
    - **Checking for proper results in all cases**
- Full verification is **not practical on large pieces of designs**, but the principles are valid across all verification.

VERIFICATION VS TEST

- Two often confused
- Purpose of *test* is to verify that the design was **manufactured** properly
- *Verification* is to ensure that the design meets the **functionality intent**

VERIFICATION AND TEST RECONVERGENCE MODEL



- Figure shows the reconvergence models for both verification and testing.
- During **testing**, **the finished silicon is reconciled with the netlist** that was submitted for manufacturing.

VERIFICATION VS TEST

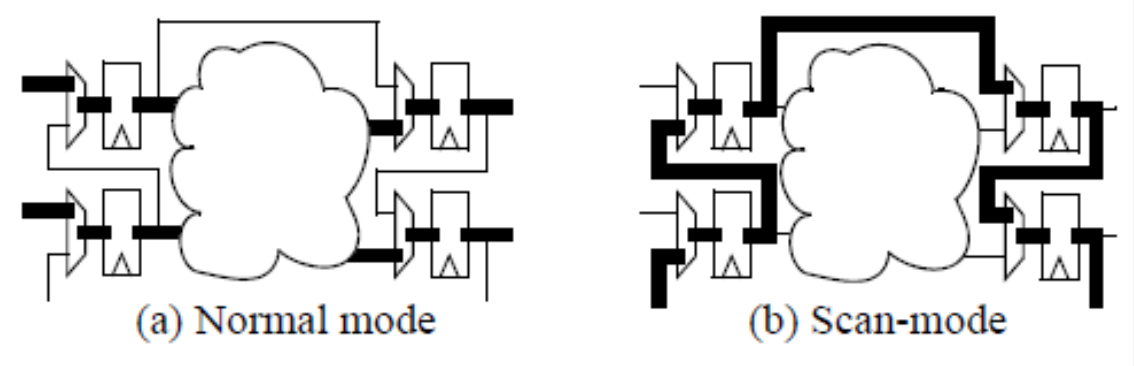
- Testing verifies that **internal nodes can be toggled**.
  - Testing is accomplished through **test vectors**.
  - The objective of these test vectors is **not to exercise functions**.
  - It is **to exercise physical locations in the design** to ensure that they can go from **0 to 1 and from 1 to 0** and that the change can be observed.
  - The **ratio of physical locations tested to the total number of such locations is called *test coverage***.
  - The test vectors are usually automatically generated **to maximize coverage while minimizing vectors** through a process called **automatic test pattern generation** (ATPG).

VERIFICATION VS. TEST

- Thoroughness of **testing depends on controllability and observability of internal nodes**.
  - Testing and test coverage depends on the **ability to set internal physical locations to either 1 or 0**, and then observe that they were indeed appropriately set.
  - Some **designs have very few inputs and outputs, but these designs have a large number of possible states**, requiring long sequences to observe and control all internal physical locations properly.
  - A perfect example is an electronic wristwatch: It has three or four inputs (the buttons around the dial) and a handful of outputs (the digits and symbols on the display).
  - However, if it includes chronometer and calendar functions, it has billions of possible state combinations (hundreds of years divided into milliseconds).
  - At speed, it would take hundreds of years to take such a design through all of its possible states.

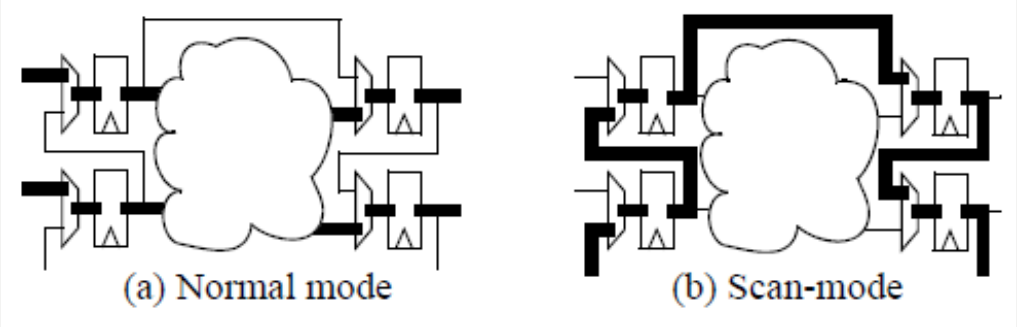
SCAN-BASED TESTING

- Linking all registers into a long shift register increases controllability and observability.
  - With scan-based tests, **all registers inside a design are hooked-up in a long serial chain**.
  - In **normal mode**, the registers operate as if the scan chain was not there.
  - In **scan mode**, the registers operate as a long shift register.



SCAN-BASED TESTING

- Linking all registers into a long shift register increases controllability and observability.
  - To test a scannable design, the unit under test is put into scan mode, then an input pattern is shifted through all of its internal registers.
  - The design is then put into normal mode and a *single* clock cycle is applied, loading the result of the normal operation based on the scanned state into the registers.
  - The design is then put into scan mode again.
  - The result is shifted out of the registers (at the same time the next input pattern is shifted in), and the result is compared against the expected value.



DESIGN FOR VERIFICATION

- Design practices need to be modified to accommodate testability requirements.
- it is acceptable to modify those same design practices to accommodate verification requirements.
- Verification must be considered during specification.
  - With functional verification requiring more effort than design, it is reasonable to require additional design effort to simplify verification.
  - Just as scan chains are put in a design to improve testability without adding to the functionality, it should be standard practice to add non-functional structures and features to facilitate verification.
  - This approach requires that verification be considered at the outset of a project, during its specification phase.

DESIGN FOR VERIFICATION

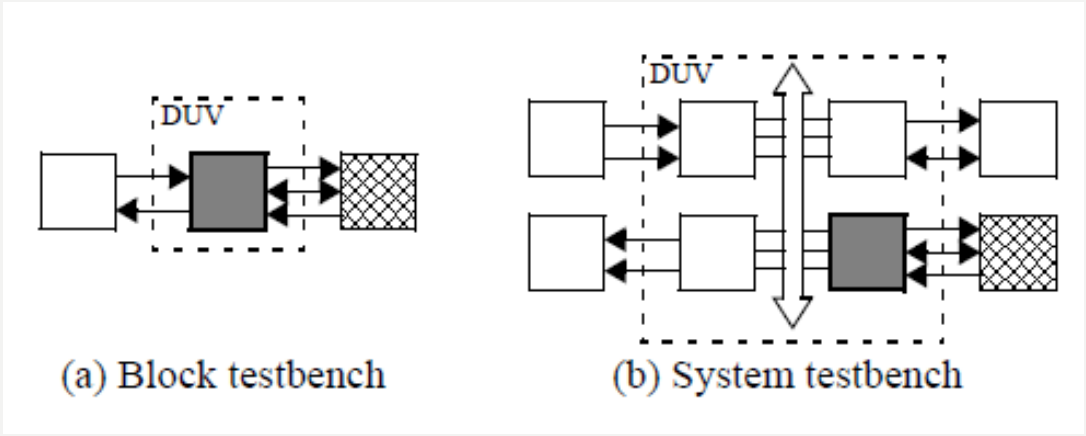
- Verification must be considered during specification.
  - Not only should the architect of the design answer the question, "What is this supposed to do?"
  - but also, "How is this thing going to be verified?"
  - Typical design-for-verification techniques include
    - well-defined interfaces,
    - clear separation of functions in relatively independent units,
    - providing additional software-accessible registers to control and observe internal locations
    - providing programmable multiplexers to isolate or bypass functional units.

DESIGN AND VERIFICATION REUSE

- Today, design reuse is a fact of life.
- It is the **best way to overcome productivity gap**
  - the difference between the **number of transistors that can be manufactured** on a single chip and
  - the **number of transistors engineers can take advantage** of in a reasonable amount of time.
  - This difference is called the *productivity gap*
  - the *productivity gap – manufactured output vs design effort per unit time*
  - **Design reuse can decrease time taken by engineer and avoid design from scratch**
- Design reuse was originally thought to be a simple concept that would be easy to put in practice.
- The reality proved—and continues to prove—to be more problematic.

VERIFICATION REUSE

- Testbench components can be reused also
  - Just like design, testbenches can also be reused
  - Figure (a) shows block level testbench
  - Figure (b) shows system level testbench using BFM (Bus Function al Model for block in Fig (a).



VERIFICATION REUSE

- Verification reuse has its challenges.
  - degrees of verification reuse,
    - some easier to achieve and some difficulties
    - similar to design reuse.
  - Reusing BFMs for the same design
    - a simple process of properly architecting a verification environment
  - Reusing testbench components or test cases in subsequent revision of the same design
    - verification of the new features difficult
  - Reusing a testbench component
    - between two different projects or between two different levels of abstraction
    - challenges must be addressed when designing the component itself

VERIFICATION REUSE

- Block and system-level testbenches **put different requirements on a BFM**
  - Block-level verification
    - **Must understand FSMs and decoders used** in implementing the interface protocol.
    - **requires a transaction-level BFM** with detailed controls of the protocol signals to vary timing or inject protocol errors.
  - System-level verification
    - **must understand the high-level functionality** that resides behind the interface block.
    - **requires the ability to encapsulate high-level data** onto the interface transactions.
    - desired level of controllability required at **much higher level than the signal-level**

THE COST OF VERIFICATION

- Verification is a necessary evil
- It always **takes too long and costs too much**
- Verification does not directly generate a profit or make money
  - it is the design being verified that will be sold and ultimately make money, not the verification.
- Yet verification is indispensable
- To be marketable and create revenues
  - a design must be functionally correct and provide the benefits that the customer requires.

THE COST OF VERIFICATION

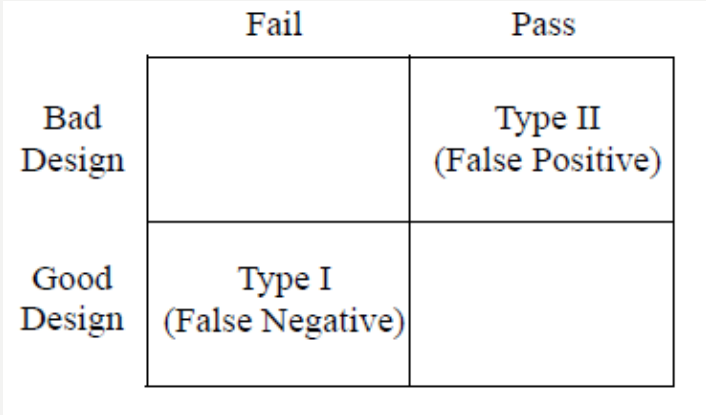
- As the number of errors left to be found decreases, the time—and cost—to identify them increases.
  - Verification is a process that is never truly complete.
  - The objective of verification is to ensure that a design is error-free, yet one cannot prove that a design is error-free.
  - Verification can show only the *presence* of errors, not their *absence*.
  - Given enough time, an error *will* be found.
  - The question thus becomes: **Is the error likely to be severe enough** to warrant the effort spent identifying it?
  - As more and more time is spent on verification, fewer and fewer errors are found with a constant incremental effort expenditure.
  - As verification progresses, it has diminishing returns.
  - It **costs more and more to find each remaining error**.

THE COST OF VERIFICATION

- Functional verification is similar to statistical hypothesis testing.
- The hypothesis under test is: "*Is my design functionally correct?*"
  - The answer can be either *yes* or *no*.
  - But either answer could be wrong.
  - These wrong answers are Type I and Type II mistakes

THE COST OF VERIFICATION

- Figure shows where each type of mistake occurs.
- Type I mistakes, or *false negatives*, are the easy ones to identify.
  - The verification is finding an error where none exist.
  - Once the misinterpretation is identified, the implementation of the verification is modified to change the answer from "no" to "yes,"
  - and the mistake no longer exists.



Type I & II mistakes

- Type II mistakes are the most serious ones:
  - The verification failed to identify an error.
  - Type II mistake, or *false positive* situation,
  - a bad design is shipped unknowingly, with all the potential consequences that entails.
  - False positives must be avoided.
- Shipping a bad design may result in simple product recall or in the total failure of the product